

pyArgus

Code and mathematical methods

STATUS	OWNER	LAST UPDATED
Engineering reference	Bryon	September 17, 2026

Authors	Prepared for Bryon by Codex
Reviewers	Technical review pending
Related docs	Source snapshot 34dd561 and repository validation records
Scope	Implementation, mathematical methods, coding process, evidence and sources

Contents

1 Purpose and evidence	3
2 System boundary and current capability	3
3 Architecture and data flow	4
4 Coding and change control process	6
5 Files and explicit contracts	7
6 Coordinate units and height systems	8
7 Time alignment and multi-file matching	9
8 Rotations and direct georeferencing	9
9 Quality assessment mathematics	10
10 Ground classification by morphology	11
11 Scaling ground classification and QA	12
12 Strip observations and linearization	13
13 Robust solution and observability	13
14 Time dependent drift and control strategy	14
15 Surfaces breaklines and contours	15
16 Above ground classification and model files	16
17 Camera projection and image colorization	17
18 AT referenced fitting and the SH 151 boundary	18
19 Viewer cross sections and workflow state	18
20 Persistence failures and validation	19
21 Recorded reference evidence	20
22 Developer operating procedure	21
23 Remaining engineering work	22
24 Glossary	23
25 Sources and principle attribution	25
Appendix A Implementation map	27
Appendix B Test map	32

The contents and page numbers are generated from the section headings. Equations are native Word math and remain editable.

1 Purpose and evidence

pyArgus is a developing Python LiDAR workstation intended to grow into an end-to-end open-source alternative to the TerraSolid suite. It already contains useful implementations of project import, quality assessment, ground and above-ground classification, strip adjustment, surface construction, contours, image colorization, and three-dimensional inspection. A completed algorithm, a passing software test, and an independently accepted survey deliverable are different milestones. This manual keeps those distinctions visible.

This document is prepared for Bryon and for developers who need to understand, maintain, or independently review the system. It describes the isolated Codex checkout at commit 34dd561, dated September 16, 2026. The manuscript date is September 17, 2026. Implementation statements refer to that snapshot; newer branches or upstream Claude Code work may differ. No production data was changed or reprocessed to prepare this document.

The authoritative implementation is the code. Historical files such as HANDOFF.md and reference/RESULTS.md explain decisions and recorded experiments, but some entries describe earlier states. For example, an early note says that all roadmap phases are complete, while the later capability audit identifies substantial product and acceptance gaps. This manual interprets phase completion as delivery of an initial capability, not TerraSolid parity.

The latest recorded validation for this checkout contains 390 passing tests, zero skipped tests, a successful dependency check, and a successful packaged executable self-test. These checks demonstrate regression coverage under the recorded environment. They do not establish accuracy for every sensor, site, or coordinate convention. Real-data results quoted later are explicitly attributed to their earlier reference runs rather than presented as new measurements.

References use [S01] through [S20] for external principles and library documentation. Code anchors name repository-relative files and functions, which can be resolved against the checkout path in Section 2. The code itself supplies the evidence for pyArgus-specific thresholds, formulas, defaults, and refusal behavior. External sources explain the underlying principles; they do not imply endorsement or identical implementation.

Reading paths

For project oversight, read Sections 2 through 5, 17 through 20, and the implementation map. For mathematical review, read Sections 6 through 16 in order. For maintenance, combine the equations with the named functions, relevant tests, and the serial validation procedure. Formula symbols are defined where first used. Positions and linear thresholds are in explicitly declared map units unless a conversion is stated; navigation angles are radians inside the mathematical core.

2 System boundary and current capability

The active checkout is C:/Users/bjordan/OneDrive - Platinum Geomatics/pyArgus-Codex. Its branch is codex/isolated-improvements. The other development checkout is C:/Users/bjordan/OneDrive - Platinum Geomatics/Desktop/pyArgus. The Codex worktree permits independent commits and its own Python environment without changing the other checkout's source files. Git worktrees share repository metadata, so branch operations and validation still require coordination.

The current desktop executable is dist/pyArgus/pyArgus.exe under the active checkout. It is a folder distribution: the adjacent _internal directory is part of the application. The source package is pyargus, the installed console entry point is pyargus.cli:main, and the package version is 0.1.0. The version string alone is insufficient to identify a development build; retain the commit, working-tree fingerprint, dependency environment, and binary hash.

Capability	Present implementation	Important boundary
Project import	Multiple LAS/LAZ and TRJ/SBET files; saved references; timestamp and strip matching	No automatic invention of CRS, clock, sensor calibration, or attitude conventions
Quality assessment	Density, strip differences, control comparisons, reports, large-project disk-backed statistics	A report is evidence to review, not automatic project certification
Classification	SMRF-style ground filter; tiled ground workflow; trained random forest for above-ground classes	Site transfer and class quality require validation; many desktop tasks remain single-cloud
Adjustment	Shared boresight perturbation, strip offsets, optional vertical drift, control observations	Delivered-cloud correction; not raw GNSS/IMU trajectory estimation
Surfaces	Mean-ground DTM, maximum-return DSM, linear TIN, soft breaklines, contours	No guaranteed constrained triangulation or complete hydro-enforcement workflow
Imagery	Oriented-image projection, calibrated lens model, RGB sampling and occlusion checks	Does not create a dense photogrammetric surface or solve aerial triangulation
Desktop	One 3D viewer, file sidebar, scoped tasks, history, QA review and full-file cross-sections	Sampled display is distinct from full-data analysis; no complete production editing suite

TerraSolid is a product objective and a functional comparison, not an implementation source. This repository does not contain TerraMatch's proprietary solver or POSPac's internal navigation processing. The mathematics of georeferencing, surface matching, robust least squares, and interpolation is publicly established. Matching the breadth, throughput, editing depth, and acceptance record of a mature suite remains a long-term engineering program.

The most consequential distinction is between adjusting a delivered cloud and rebuilding one from raw observations. pyArgus can attach a compatible trajectory to already georeferenced points and estimate additional corrections. Recomputing an original cloud rigorously may also require raw ranges, scan angles, time synchronization, lever arms, instrument calibration, GNSS observations, IMU measurements, and the original processing conventions. An SBET and a LAS file do not necessarily recover that information.

3 Architecture and data flow

The architecture separates numerical methods, file-format adapters, orchestration, and interface state. NumPy and SciPy form the computational base. Optional adapters use laspy/lazrs for LAS/LAZ, pyproj for coordinate transformations, scikit-learn and joblib for learned classification, Pillow for imagery, and ezdxf for CAD input.

Optional imports are delayed so the mathematical core can be used without every desktop or format dependency installed.

Layer	Main location	Responsibility
Numerical foundations	pyargus/core	Units, rotation matrices, direct georeferencing and grid operations
Algorithms	pyargus/qa, align, classify, surfaces, imagery	Compute results from arrays and explicit parameters
File boundaries	pyargus/formats	Parse, validate, transform and write external representations
Jobs and projects	project.py, large_qa.py, analysis_records.py, job_manifest.py	Match inputs, manage work, capture provenance and publish outputs
Desktop and review	gui.py, workspace_gui.py, viewer3d.py, review_gui.py, sections.py	User input, task scope, visualization, review and cancellation
Evidence	tests, reference, devtools, packaging	Synthetic regression, field references, validation records and builds

The typical data flow is: register files; validate identities, schemas, CRS and clocks; inspect and match; generate baseline QA; classify when needed; solve and apply a justified correction; reopen exported results for QA; create surfaces, contours or colors; inspect the results; record acceptance and delivery selections. The operator may legitimately change that order for already classified or previously adjusted inputs, but the history must record what was actually done.

Figure 1 is the processing dependency chart expressed as an auditable mapping. The review columns are deliberate: mathematical outputs do not bypass the acceptance step.

Inputs	Computation	Stored evidence	Review before continuation
Clouds and trajectories	Inventory and unique matching	Project settings and match counts	Coverage, unresolved matches, conventions
Classified clouds and control	Density and residual statistics	QA report and job record	Ground suitability and residual patterns

Inputs	Computation	Stored evidence	Review before continuation
Ground overlaps and trajectory geometry	Robust strip adjustment	Parameters, corrected clouds, before and after QA	Independent checkpoints and spatial behavior
Accepted cloud and optional breaklines	DTM, DSM, TIN and contours	Surface and vector files	Gaps, seams, breaklines and terrain fidelity
Cloud, EO, calibration and photographs	Projection and color sampling	RGB cloud and camera provenance	Calibration consistency, coverage and occlusion

The desktop shares processing jobs with command-line workflows rather than implementing alternate mathematics. A background runner performs long operations while Tk owns the user-interface thread. Queues carry progress, results and failures back to the interface. Cancellation is cooperative at defined checkpoints; it is not a promise that every numerical solve can stop immediately.

4 Coding and change control process

Development begins with a concrete data contract and a falsifiable result. A new algorithm needs specified input shapes, units, coordinate frames, time bases, output semantics, failure cases, and an independent way to check its answer. The intended numerical change belongs in the algorithm layer; user-interface code should collect settings, invoke that implementation, and display its result.

The repository's recurring development pattern is to construct a small synthetic scene with known truth, implement the simplest justified method, test boundary and adversarial cases, and then exercise an applicable read-only field reference. Where a defect appears, the regression should reproduce the failure rather than merely repeat the implementation. Examples include heading wrap across pi, degenerate strip geometry, large State Plane coordinate magnitudes, camera distortion after quarter-turn image rotation, and a building crossing a ground-classification tile seam.

Shared policy prevents different entry points from disagreeing. `classify/job.py` holds the return eligibility and labeling rules used by whole-cloud and tiled ground classification. The common imagery job performs calibration selection and provenance checks for both command line and desktop. Project QA and alignment use common recording adapters. Fixing only a GUI path while leaving the CLI different would defeat this design.

Step	Coding action	Evidence to retain
Define	State the behavior, assumptions and accepted domain	Issue or design note with a measurable outcome
Implement	Keep numerical functions independent of widgets and file dialogs	Reviewable changes and explicit parameter contracts
Challenge	Test sign, units, boundaries, malformed inputs and invariants	Regression that fails for the original defect

Step	Coding action	Evidence to retain
Validate	Run focused checks, then required full and reference gates serially	Logs, JUnit results and validation.json
Package	Build a fresh executable distribution and exercise it	Build log, binary hash and packaged self-test
Review	Compare exported products with independent evidence	Acceptance notes tied to inputs and version

The workstation has a recorded history of MSYS fork failures under concurrent heavy shells. Heavy tests, reference runs and packaging therefore run serially. devtools.validate uses a shared Git-directory lock to prevent overlapping runs by cooperating worktrees. A direct pytest command does not honor that lock, so concurrent agents still need coordination.

Source datasets on Z: are reference inputs. New results belong in distinct output locations. Destructive source replacement is not the normal processing path. Documented assumptions must be revisited when changing sensor, platform, export software, calibration, coordinate system, or flight geometry. A convention validated on Summerville's TrueView 660 data is not automatically established for the fixed-wing SH 151 dataset.

5 Files and explicit contracts

LAS coordinates are quantized integers with per-axis scale and offset. The working arrays expose decoded positions; outputs must respect the original schema and quantization. Point source IDs are not globally unique identifiers by themselves. A project can contain tiles of the same flight or unrelated flights reusing the same ID. pyArgus therefore builds analysis identities from project associations while preserving original point_source_id values in exported source files. [S01, S17]

$$X = I_x s_x + o_x \quad Y = I_y s_y + o_y \quad Z = I_z s_z + o_z \quad (1)$$

Here I is the stored integer, s the scale and o the offset. Rounding to a quantization step q can introduce approximately q/2 error in a representable coordinate. A very small reported adjustment may disappear on export; post-export verification must use the stored result rather than only the floating-point array held by a solver.

SBET input uses fixed little-endian records of 17 doubles, or 136 bytes. Time, latitude, longitude, altitude, velocity, attitude and inertial terms have defined positions. Angles are radians and altitude is ellipsoidal meters. The format alone does not establish the geographic reference frame of every vendor export. The current coordinate adapter assumes NAD83(2011), which must be verified for the supplied file. The reader refuses incomplete record sizes and non-increasing timestamps. [S02]

Native TerraScan input is the specific TSCANTRJ layout version 20010715: a 1376-byte header and 64-byte position records. The parser checks recognition bytes, version, stride, count, finite coordinates and angles, increasing time and agreement between header and record time bounds. Native XYZ values are retained. TRJ angles are exposed as degrees at the file boundary; the common alignment loader converts conventions for the core. The binary layout does not establish the project CRS, elevation datum or stored clock. [S03]

Control CSV column order must be explicit. PNEZ means point identifier, northing, easting, elevation; PENZ means identifier, easting, northing, elevation. Inside the computational model, X is easting and Y is northing. A plausible-

looking pair of coordinates is not an adequate reason to guess the order. Ground control used to estimate an adjustment must be distinguished from checkpoints reserved to evaluate it.

EO files supply image names, camera positions and orientation information. Camera calibration supplies image dimensions, focal length, principal-point offsets and distortion coefficients. pyArgus's implemented .cal interpretation is specific; arbitrary P4D or vendor coefficient files are not interchangeable merely because they contain familiar variable names. Matching physical camera roles and calibration authority remains part of project preparation.

Code anchors: `formats/las.py`; `formats/sbet.py`; `formats/trj.py`; `formats/trajectory.py`; `formats/control.py`; `formats/eo.py`; `imagery/camera.py`. Tests: `test_las`, `test_sbet`, `test_trj`, `test_control`, `test_calibration_selection` and `test_camera_consistency`.

6 Coordinate units and height systems

Linear units are mathematical inputs. One international foot is exactly 0.3048 meter; one US survey foot is exactly 1200/3937 meter. The distinction is about two parts per million and can create several feet of discrepancy at large coordinate magnitudes. NIST identifies the survey foot as a legacy unit; existing datasets must still be interpreted according to the units in which they were created, rather than silently relabeled. [S04]

$$1 \text{ ft}_{US} = \frac{1200}{3937} \text{ m} \quad 1 \text{ ft}_{int} = 0.3048 \text{ m} \quad (2)$$

Horizontal projection and vertical datum conversion are separate requirements. An SBET height h is normally ellipsoidal. An orthometric elevation H depends on geoid undulation N . For the implemented constant- N path, N is supplied in meters and the elevation is converted to the target height unit after subtraction. [S05]

$$H = h - N \quad H_{map} = \frac{h-N}{u} \quad (3)$$

The factor u is meters per target elevation unit. If h is 200 m and N is -29 m, the orthometric value is 229 m, not 171 m. This is an illustrative calculation, not a site measurement. The horizontal transformer uses explicit axis order. A compound target CRS may have a distinct vertical unit; code must not assume that its height unit equals its horizontal unit.

For a specified vertical CRS, `formats/crs.py` composes the appropriate target and calls `pyproj` with `always_xy=True`, `allow_ballpark=False` and `only_best=True`. Missing geoid resources cause a refusal rather than a finite but inappropriate substitute transformation. Finite numbers alone cannot demonstrate that an orthometric transformation was performed. [S06]

A constant geoid undulation is an available approximation, not a universal production recommendation. Its adequacy depends on project extent, relief, geoid variation, the required accuracy, and the chosen datum. The user must supply a justified value and sign. Projection to a common CRS also does not eliminate the need to establish whether attitudes reference true north, grid north or another navigation frame.

Important limitation: trajectory-to-cloud height checks can reject a platform below the ground, but a smaller consistent datum error can still pass. Such an error changes the effective range vector and therefore the magnitude of a boresight correction. Coordinate checks, flight-height diagnostics and independent marks must be considered together.

Code anchors: `core/units.py` and `formats/crs.py:sbet_to_map`. Tests: `test_units`, `test_crs` and `test_attach`. The reference dataset's explicit EPSG and geoid choices belong to its evidence record and must not become universal defaults for other sites.

7 Time alignment and multi-file matching

LiDAR positions and trajectory states must refer to the same instant. LAS can carry adjusted standard GPS time or GPS seconds of week. The LAS time-type flag matters. Adjusted standard GPS is full GPS seconds minus one billion; converting it to week time for a known week W uses the equation below. GPS time is not UTC, and filenames or human-readable flight dates are not substitutes for the clock declaration. [S01]

$$t_{week} = t_{LAS} + 10^9 - 604800 W \quad (4)$$

The recorded Summerville join used week 2385 and placed 100 percent of returns inside its SBET interval. That is a recorded dataset-specific result. The older single-cloud helper estimates a week from median offsets and refuses obvious seconds-of-week input. The multi-file project path additionally validates LAS clock declarations and requires project-week information where necessary. Native TRJ offers explicit same-stored-time or GPS-week-time modes.

For interpolation between trajectory epochs t_0 and t_1 , a position or nonwrapping scalar uses a linear weight. Heading is first unwrapped, then interpolated, then wrapped back to the angular interval. Direct interpolation between headings near $+179$ and -179 degrees would incorrectly pass through zero rather than through the nearby 180-degree direction.

$$w = \frac{t-t_0}{t_1-t_0} \quad p(t) = (1-w)p_0 + wp_1 \quad (5)$$

Project matching streams LAS chunks and tests candidate trajectory coverage, configured gaps, native line identifiers and explicit file bindings. It reports matched, unmatched and ambiguous returns separately. Inventory may succeed while revealing incomplete coverage; QA and alignment with supplied trajectories require complete unique association. A matched timestamp is necessary but not sufficient evidence that the trajectory is physically correct.

Interpolation must not bridge different files or an internal outage larger than the configured gap. Overlapping SBET intervals can require explicit cloud-to-trajectory bindings. Repeated source line IDs without trajectories require an explicit declaration that they describe shared flights. Analysis strips are therefore neither synonymous with files nor safely identified by an unqualified integer line number.

During project processing, input size and modification times are checked for changes. Those checks detect ordinary replacement or modification; they are not cryptographic proof of unchanged contents. The inventory keeps file and trajectory counts, extents, associations and diagnostics so a later reviewer can reconstruct the selection.

Code anchors: `project.py:_cloud_headers`, `_read_tracks`, `_point_times`, `_coverage` and `load`;
`formats/sbet.py:week_alignment` and `interpolate`. Tests: `test_project`, `test_sbet`, `test_trj` and `test_attach`.

8 Rotations and direct georeferencing

The core uses a stated rotation convention: R maps body-frame vectors to the local mapping frame and is composed as yaw, pitch, then roll in matrix notation. The angle inputs are radians. Columns and multiplication direction matter; the inverse of an orthonormal rotation is its transpose. `core/rotation.py` provides scalar and vectorized construction, and an inverse angle extraction that refuses the gimbal poles where roll and yaw cannot be separated.

$$R(\varphi, \theta, \psi) = R_z(\psi)R_y(\theta)R_x(\varphi) \quad R^{-1} = R^T \quad (6)$$

The forward georeferencing model combines navigation position, navigation attitude, scanner-to-body boresight rotation, scanner range vector and lever arm. Here X is a ground point, P is the navigation origin, b is a scanner-frame vector, l is the IMU-to-scanner lever arm in the body frame, and R_b is the fixed boresight rotation. The range vector already includes direction and distance.

$$X = P + R(R_b b + l) \quad (7)$$

This is a physical forward model implemented in `core/georef.py:ground_points`. It is not a full acquisition or navigation estimator. It does not independently recover clock offsets, GNSS ambiguities, IMU bias, scanner range bias or the original raw beam measurement. Literature on strip adjustment explains why overlap geometry can help recalibrate a sensor system, but `pyArgus` uses a smaller implemented parameterization. [S07]

To work with an already georeferenced cloud, `align/bundles.py` constructs an effective nominal body vector from the delivered point and attached navigation state. It keeps that vector fixed while evaluating first-order corrections. The effective vector may embody effects already applied by the upstream processing system; it is not guaranteed to equal the original raw scanner vector.

$$b_i = R_i^T (X_i - P_i) \quad (8)$$

$$X_i' = X_i + R_i(\beta \times b_i) + o_s \quad (9)$$

The small vector β contains shared boresight perturbations and o_s is the offset for strip s . The cross product is a first-order approximation. For an illustrative 100 m range, a 0.001 rad angle produces a displacement on the order of 0.1 m. The omitted second-order scale is on the order of range times angle squared. Larger corrections require reviewing the approximation and the upstream model rather than simply trusting a converged solve.

The SBET attachment adapter maps its stated aerospace convention to roll, negative pitch, and $\pi/2$ minus heading. It compares heading and heading-plus-wander with the transformed flight track and reports the selected candidate and errors. This empirical check can detect gross mistakes, but aircraft crab, map convergence and new vendor conventions require separate scrutiny. TRJ geometry requires explicit convention confirmation before alignment.

Tests: `test_rotation`, `test_georef`, `test_attach` and `test_align`. Verify a known axis vector, an inverse round trip, an angular-wrap example and a controlled perturbation before applying the mapping to a new sensor.

9 Quality assessment mathematics

Density is count divided by cell area. A cell size c in map units produces density in points per squared map unit; conversion to points per square meter requires the square of the linear unit factor. Occupied-cell distributions describe coverage and overlap structure, not necessarily a defect. Empty coverage and project footprint interpretation must remain explicit.

$$\rho_{ij} = \frac{n_{ij}}{c^2} \quad (10)$$

Strip QA compares median elevations in common ground cells. Its residual sign is B minus A. Checkpoint residuals instead use LiDAR surface minus surveyed elevation. Positive checkpoint residual means the cloud is above the reference. Neither residual should be confused with the sign of the correction that must be added to the cloud.

$$d_{AB,j} = \text{median}(Z_{B,j}) - \text{median}(Z_{A,j}) \quad (11)$$

$$r_k = Z_{\text{surface},k} - Z_{\text{check},k} \quad (12)$$

Median cell differences reduce sensitivity to isolated points but are not identical homologous-surface measurements. On a slope, differing sample locations within a cell can create a vertical difference even if both strips are correctly positioned. Vegetation, roofs, moving objects and ground-label errors can also contaminate overlap evidence. Ground-only statistics and cross-sections are useful controls on interpretation.

The local checkpoint method uses the median of ground returns inside a circular XY neighborhood and requires a minimum count. Marks without adequate local returns are reported as skipped with a reason. The TIN method linearly interpolates a surface at the mark and produces NaN outside the convex hull. The two methods answer

different questions, especially near a curb or ditch; they must not be mixed in a comparison without naming the method.

$$RMSE_z = \sqrt{\frac{1}{n} \sum_{k=1}^n r_k^2} \quad (13)$$

$$NMAD = 1.4826 \text{ median}(|r_k - \text{median}(r)|) \quad (14)$$

The implemented `asprs_vertical` helper returns count, mean, RMSEz, 1.96 times RMSEz and the 95th percentile of absolute residual. Those latter interpretations follow the repository's 2014-era convention. The newer ASPRS Edition 2 Version 2 changes accuracy reporting, including removal of the 95-percent confidence-level accuracy measure. Do not label this helper a complete implementation of the newer standard. Acceptance also requires the applicable checkpoint distribution, independence, survey uncertainty and project specifications. [S08, S09]

A worked sign example: if the ground estimate is 100.18 ft and the checkpoint is 100.00 ft, the QA residual is +0.18 ft. A simple elevation-only correction to meet that mark is -0.18 ft. A zero mean residual can coexist with a large RMSE when positive and negative errors cancel.

Code anchors: `qa/density.py`; `qa/overlap.py`; `qa/checkpoints.py`; `qa/control_by_strip.py`; `qa/report.py`. Tests cover grid boundaries, skipped marks, sign conventions and known residual distributions.

10 Ground classification by morphology

The ground filter is a NumPy/SciPy implementation based on the Simple Morphological Filter family. The cited 2013 paper is by Thomas J. Pingel, Keith C. Clarke and William A. McBride. An older repository docstring names Nelson; that attribution is incorrect. PDAL documents a related SMRF implementation, but `pyArgus` does not call that filter and its code should not be assumed to match every PDAL option or result. [S10, S11]

The first step grids the lowest candidate elevation in each XY cell. Empty cells are filled from the nearest known cell to obtain a provisional working surface. Candidate eligibility is shared across whole-cloud and tiled jobs: by default only last returns can establish or be labeled ground. The optional any-return policy changes that decision explicitly. The low-level filter classifies the points supplied to it; the job layer owns eligibility.

$$S_0(i, j) = \min_{p \in \text{cell}(i, j)} Z_p \quad (15)$$

A morphological opening is erosion followed by dilation with a disk-shaped structuring element. Openings suppress elevated objects smaller than their neighborhood while retaining broader terrain. `pyArgus` applies a sequence of openings to the previous round's surface, with increasing integer cell radius. Cells whose drop exceeds the slope-dependent threshold are flagged as objects. This recurrence, rather than a one-pass filter, determines the tiling influence distance. [S20]

$$S_r = \text{opening}(S_{r-1}, B_r) \quad (16)$$

$$\text{object}(i, j) = \bigvee_r [S_{r-1}(i, j) - S_r(i, j) > a \, r \, c] \quad (17)$$

Here a is the slope threshold, r the radius in cells and c the cell size. Surviving measured minimum cells support the provisional DEM; nearest filling supplies values elsewhere. The code computes the DEM gradient and bilinearly samples both DEM and slope at each input point. A symmetric elevation test prevents points far below the surface from being accepted solely because they are low.

$$\text{ground}(p) = [|Z_p - \text{DEM}(p)| \leq h_0 + k |\nabla \text{DEM}(p)|] \quad (18)$$

The threshold h_0 is an elevation distance; the slope magnitude is dimensionless. In this formula k also supplies a distance scale per unit slope. Converting a metric project to feet therefore requires a coherent parameter choice, not changing the cell size alone. The code's numerical defaults and the historical survey-foot reference settings are operational presets, not a proof of scale invariance.

An optional `low_cut` opens the inverted surface to reject low cells. It is intentionally off by default. Under canopy, real ground penetrations can look like pits among canopy-only cells, so indiscriminate low-cell removal can lift the DEM into the vegetation. The recorded Summerville experiment demonstrates that failure. Ground-job output labeling is principally ground class 2 versus unclassified class 1; it is not a general preservation rule for all previous class labels.

Code anchors: `classify/ground.py:ground_surface` and `classify_against`; `classify/job.py:candidates` and `labels`; `core/gridding.py`. Tests: `test_ground`, `test_tiled_ground` and `test_cli_classify`.

11 Scaling ground classification and QA

Streaming an input file does not guarantee a bounded-memory algorithm. Ground classification can build a raster tile by tile and then stream point classification against the assembled surface. Strip adjustment still constructs arrays for the selected block. The GUI's project analysis limit defaults to 25 million points for array-based work; large QA follows a separate disk-backed path. Raising an array limit is a resource decision, not a streaming implementation.

The tiled ground driver uses one global lattice. Each output tile has a surrounding halo so neighborhood operations can see relevant points beyond its core. For progressive opening radii 1 through R , one opening can propagate information across twice its radius. The code's conservative morphological cascade reach is therefore:

$$R = \lceil \frac{\text{window}}{c} \rceil \quad H_{\text{cascade}} = c \sum_{r=1}^R 2r = cR(R+1) \quad (19)$$

With $c=3$ ft and `window=60` ft, $R=20$ and the calculated reach is 1260 ft. A small nominal tile can consequently require a much larger halo box. On ordinary LAS/LAZ without a spatial index, each tile can require rereading the file. The code reports the plan and measured tile load; the assembled raster also consumes memory.

This reach explains the implemented default, but it is not a universal proof of whole-cloud equivalence for arbitrary voids. Nearest-cell filling can depend on data outside a local box, and slope evaluation introduces further boundary concerns. The tests compare difficult seam scenes, including structures crossing tile boundaries. A production acceptance should also exercise representative data gaps and project edges. Smaller user-selected halos trade assurance for memory and I/O and require direct seam checks.

COPC stores a spatial hierarchy that supports localized queries. `pyArgus` can use `laspy` to read indexed regions and an external PDAL executable to write COPC. The writer explicitly forwards metadata and extra dimensions and checks the output. COPC conversion reorders points: row-by-row comparison against the original LAS is invalid unless identity is re-established. The optional converter is an external native dependency, not part of the pure mathematical core.

Large QA uses SQLite as scratch storage and external sorting for exact cell medians and percentiles. Cells receive all contributing sources before comparisons; rendering tiles do not independently estimate their own statistics. A percentile uses the rank $(n-1)q$ with linear interpolation between adjacent ordered values, matching the intended quantile definition.

$$u = (n-1)q \quad Q(q) = z_{\lfloor u \rfloor} + (u - \lfloor u \rfloor)(z_{\lceil u \rceil} - z_{\lfloor u \rfloor}) \quad (20)$$

Disk space and throughput remain real constraints. Project guidance estimates roughly 160 scratch bytes per source point plus additional temporary space; this is planning guidance, not a universal exact bound. A 402-million-point project can therefore require tens of gigabytes of scratch even though memory remains controlled. SQLite scratch durability is deliberately relaxed because it is rebuildable work, unlike the persistent job record.

Code anchors: `classify/tiles.py`; `classify/job.py`; `formats/copc.py`; `large_qa.py`. Tests: `test_tiled_ground`, `test_streaming`, `test_large_qa` and `test_copc`.

12 Strip observations and linearization

The adjustment uses corresponding surface patches rather than assuming that the nearest individual points represent the same physical return. For each common XY cell, strip A supplies a plane fitted by singular value decomposition. Subtract its centroid from the points and take the right singular vector associated with the smallest singular value as the normal. The normal is flipped upward. Plane RMS is the smallest singular value divided by the square root of the sample count.

$$A_c = U \Sigma V^T \quad n = V_{min} \quad RMS_{plane} = \frac{\sigma_{min}}{\sqrt{N}} \quad (21)$$

A correspondence measures strip B's centroid against that plane. The default patch gates require enough samples, acceptable plane RMS, a sufficiently upward normal and a bounded separation. The low-level defaults include cell=5 map units, min_points=8, min_normal_z=0.7, max_rms=cell/25 and max_distance=cell/2. The project wrapper can use different defaults; record the actual caller settings.

$$d = n^T (\bar{X}_B - \bar{X}_A) \quad (22)$$

The patch is asymmetric: the plane is fitted to A, while B contributes its centroid and a minimum support count. It is therefore not a complete symmetric ICP implementation. The method belongs to the established surface-matching and strip-adjustment family, but the specific observation construction is pyArgus's own code. [S07]

To remove d, corrections must satisfy the following linearized equation. Each side uses a point near its centroid as an anchor for navigation state; averaging angles could introduce wrap errors. Define mA=RA transpose times n and mB=RB transpose times n.

$$n^T (\Delta X_A - \Delta X_B) = d \quad (23)$$

$$J_\beta = (m_B \times b_B) - (m_A \times b_A) \quad (24)$$

$$J_\beta \Delta \beta + n^T \Delta o_A - n^T \Delta o_B = d \quad (25)$$

For vertical-only offsets, each offset column is the normal's Z component, positive for A and negative for B. For full XYZ offsets, the row uses the three normal components. The code recomputes patch correspondences from corrected coordinates at each outer iteration while retaining the original nominal body vectors.

Control observations are point-to-plane constraints on one strip. A local plane is fitted around the surveyed mark, using the implementation's square XY neighborhood, and the residual is n transpose times mark minus centroid. Its positive sign means that the strip must move toward the mark along the normal. The default control radius is 6 map units, support is 10 points and weight is 10. This weight is a numerical relative weight, not a calibrated survey uncertainty model.

$$d_c = n^T (C - \bar{X}_s) \quad J_{\beta,c} = -(m_s \times b_s) \quad (26)$$

Code anchors: align/patches.py:correspondences and control_observations. Tests: test_align, test_control_by_strip and reference/alignment_proof.py. Independent post-correction QA is still required because a patch residual assesses the model's own observations.

13 Robust solution and observability

The solver stacks patch and control rows into a design matrix J and a right-hand side d. Unknowns can include three shared boresight components, per-strip vertical or XYZ offsets, or time-dependent vertical nodes. Without control, constant-offset mode fixes the first strip's offset to zero to remove the free datum. With usable control, the marks can anchor the absolute solution through the connected overlap network.

$$\Delta x = \operatorname{argmin}_v \sum_i w_i (d_i - J_i v)^2 \quad (27)$$

Outliers are downweighted by three inner Huber reweighting passes. The code estimates residual scale using NMAD and assigns a robust factor that is one for small residuals and decreases inversely with the magnitude of large residuals. Huber's work supplies the robust-estimation principle; the exact loop count, scale estimate and tuning value here are implementation choices. [S12]

$$s = 1.4826 \text{ median}(|r - \text{median}(r)|) \quad w_{\text{robust},i} = \min(1, \frac{1.345s}{|r_i|}) \quad (28)$$

Base weights multiply the robust factors. Control rows receive the configured control weight. Drift stiffness and gauge rows are not robustly downweighted, because weakening the regularizer precisely where a curve departs most would undermine its purpose. When the scale is nonpositive, the implementation stops reweighting rather than dividing by zero.

Angles and distances have different numerical scales. The solver weights the rows, normalizes each column, evaluates the condition of the scaled normal matrix, solves in that same scaled space, and then converts the update back. This avoids allowing units alone to dominate the observability diagnosis.

$$A = W^{1/2} J \quad D_{jj} = \|A_{:,j}\|_2 \quad \tilde{A} = AD^{-1} \quad (29)$$

$$\Delta x = D^{-1}(\tilde{A}^T \tilde{A})^{-1} \tilde{A}^T W^{1/2} d \quad (30)$$

The equation shows the mathematical normal-equation solution; the implementation uses `numpy.linalg.solve` rather than explicitly forming an inverse. Zero-leverage columns cause refusal. The constant-system condition threshold is 200. This is a project-specific numerical gate supported by tests, not a universal geodetic standard or a confidence interval. Forming normal equations also squares the design matrix condition number; a future sparse QR or SVD solution is a reasonable engineering improvement.

Observability is physical as well as numerical. Parallel flat strips can leave yaw unobservable. Boresight pitch can resemble a vertical trend along a line; opposing headings, crossing lines and terrain relief can help separate effects. A small residual is not sufficient if multiple parameter combinations produce it. The default outer loop permits eight iterations and stops when the largest numerical update falls below 1e-10; that mixed-unit stopping rule is not a survey accuracy tolerance.

Code anchor: `align/solve.py:solve_alignment` and `_constant_observability_gate`. Tests include known injected offsets, recoverable boresight, flat-scene degeneracy and control anchoring. The current result is not a complete parameter covariance or propagated point uncertainty product.

14 Time dependent drift and control strategy

Constant offsets cannot represent a vertical error that varies during a flight line. Drift mode replaces each strip's constant vertical offset with a piecewise-linear correction over timestamp nodes. A point uses the two nodes that bracket its time; alpha below is the interpolation weight, not the camera or correction-fit parameter with the same conventional name elsewhere.

$$\alpha = \frac{t - t_k}{t_{k+1} - t_k} \quad z_{\text{corr}}(t) = (1 - \alpha)c_k + \alpha c_{k+1} \quad (31)$$

Node spacing is configured in seconds. Even a short nonzero-duration strip receives at least two nodes. The core requires per-point times, positive spacing and positive stiffness. Drift is vertical-only; it does not estimate an independent full three-dimensional navigation error history.

Smoothness rows penalize differences between adjacent accumulated node values. Their coefficient is stiffness divided by the square root of the time interval. Squaring those rows approximates the integral of squared correction rate, so changing node spacing does not simply dilute the intended smoothing strength.

$$P = \lambda^2 \sum_k \frac{(c_{k+1} - c_k)^2}{t_{k+1} - t_k} \quad (32)$$

The right-hand side includes the current penalty residual. Penalizing only each iteration's increment would allow the accumulated curve to escape its prior after enough iterations. The repository records this distinction as a previously identified failure mode. Without control, drift mode constrains the first strip's node mean to zero, but overlap differences still cannot uniquely establish a shared time-varying absolute motion.

For example, if two strips differ by a slowly varying 0.20 ft error, a relative solution may split the correction between them. Near-zero final strip difference does not prove which strip was originally wrong. Checkpoints or trustworthy fixed geometry are needed when the absolute allocation matters. The core's drift interpolation clamps outside its node range; callers should keep evaluation within the modeled time support rather than interpreting that behavior as justified extrapolation.

Drift regularization can make an ill-observed system appear numerically well-conditioned. pyArgus therefore tests a nested constant-offset system for observability before solving the regularized drift system. Its additional drift sanity gate is $1e12$. The two thresholds serve different purposes and should not be compared as interchangeable acceptance criteria.

Boresight and drift are coupled: an along-track angular effect can be absorbed by a smooth vertical curve. The recorded development guidance is to calibrate boresight on clean geometry and then evaluate drift with that calibration fixed. This is a methodological recommendation supported by the repository's experiments, not a claim to reproduce TerraMatch internals. The core supports drift options that are not all exposed in every project GUI path.

Code anchors: `align/drift.py` and `align/solve.py`. Tests: `test_drift` and the synthetic reference/`alignment_proof.py` experiments. Reserve independent marks before fitting rather than evaluating only the control points that determined the curve.

15 Surfaces breaklines and contours

The gridded DTM is the mean elevation of supplied ground points in each cell. The DSM is the maximum elevation of all supplied returns. Empty cells can be filled from the nearest valid cell only within `max_fill` cells; more distant cells remain nodata. These are clearly defined estimators, not interchangeable reconstructions of a continuous physical surface.

$$DTM_j = \frac{1}{n_j} \sum_{i \in j} Z_i \quad DSM_j = \max_{i \in j} Z_i \quad (33)$$

Grid samples used for interpolation sit at cell centers. Bilinear interpolation combines the four neighboring samples with fractional XY weights. Internally grids are indexed as `[ix,iy]`; exported raster rows are transposed and reversed to be north-first. ESRI ASCII output requires square cells and writes elevations to three decimal places, which introduces an additional output rounding consideration.

$$z(u, v) = (1 - u)(1 - v)z_{00} + u(1 - v)z_{10} + uvz_{11} + (1 - u)vz_{01} \quad (34)$$

A TIN uses SciPy's Delaunay-based linear interpolation. Inside a triangle, barycentric weights sum to one and interpolate its three vertex elevations. Outside the convex hull the result is NaN. A convex hull alone does not describe internal holes or guarantee reasonable triangle lengths across sparse regions. [S13]

$$z = \lambda_1 z_1 + \lambda_2 z_2 + \lambda_3 z_3 \quad \lambda_1 + \lambda_2 + \lambda_3 = 1 \quad (35)$$

Breaklines are densified using XY segment length, retaining original vertices and interpolating Z linearly along each segment. Their vertices join the surface point set. This is soft enforcement: a triangle edge is not guaranteed to coincide with the breakline between samples. Conflicting XY/Z vertices, hard discontinuities and hydro-enforced edges need explicit handling and a constrained triangulation design before stronger guarantees can be made.

Contours are extracted from the cell-center grid with marching squares. Each valid square is classified by which corners lie above a level. Edge crossings are linearly interpolated; saddle cases use the code's cell-average decision. Squares touching nodata are skipped. Segments are joined into lines, with endpoint keys rounded to nine decimal places, and short chains can be discarded according to `min_vertices`.

$$t = \frac{L - z_a}{z_b - z_a} \quad P_L = P_a + t(P_b - P_a) \quad (36)$$

The contour has elevation L by construction on the interpolated edge. It is not thereby proven to follow the true terrain between points. Optional Chaikin smoothing cuts corners at quarter and three-quarter positions. Smoothing changes XY and can move the line off its source surface, so it is off by default and must be disclosed when used.

Code anchors: `surfaces/dtm.py`, `tin.py` and `contours.py`; `formats/breaklines.py`, `dxs.py` and `geojson.py`. Tests cover known planes and cones, hull behavior, nodata termination, elevation preservation and breakline densification. Constrained breaklines, hydrologic topology and surface editing remain future production work.

16 Above ground classification and model files

Ground classification and above-ground classification use different methods. The former is a geometric filter; the latter uses a random forest trained on labeled examples. Its feature vector is height above ground, planarity, linearity, sphericity, cell height range, cell population, return-number ratio and total return count. Neighborhoods are grid cells, not nearest-neighbor spheres, which makes the computation economical for dense clouds.

Cell shape comes from a centered covariance matrix. The implementation first computes cell means, then accumulates products of deviations. Computing mean of squares minus square of mean directly would lose precision at large State Plane coordinate magnitudes. Eigenvalues are ordered largest to smallest and negative numerical remnants are clamped to zero.

$$C = \frac{1}{n} \sum_i (p_i - \bar{p})(p_i - \bar{p})^T \quad \lambda_1 \geq \lambda_2 \geq \lambda_3 \geq 0 \quad (37)$$

$$linearity = \frac{\lambda_1 - \lambda_2}{\lambda_1} \quad planarity = \frac{\lambda_2 - \lambda_3}{\lambda_1} \quad sphericity = \frac{\lambda_3}{\lambda_1} \quad (38)$$

Cells with fewer than three points receive zero shape features. A numerical denominator floor prevents division by zero. Height above ground is point elevation minus the sampled DTM; points outside ground coverage or beyond valid filling support remain unclassifiable. Noise must be excluded from feature statistics, not merely relabeled after prediction, because one outlier can contaminate the neighboring cell's shape.

The training function caps the number of examples per class, uses a reproducible random seed, and constructs 100 trees by default, with minimum leaf size 5 and `balanced_subsample` class weighting. A forest combines multiple randomized decision trees; the actual scikit-learn predictor averages class probabilities across trees before selecting a class. These values are code defaults, not independently optimized settings for every flight. [S14, S15]

Training creates the `.joblib` model; a user does not need to manufacture that file before training. The saved object includes the forest, feature names, class labels, notes and additional metadata such as feature cell and coordinate units where populated. Loading verifies feature compatibility and the scikit-learn major version. That limited guard is not a guarantee of compatibility across all dependency versions.

Joblib uses pickle-style persistence and should only load trusted model files. It is not a neutral interchangeable geospatial format, and loading an untrusted file can execute code. This is a practical input constraint of the implemented workflow, not a judgment about the model's classification accuracy. Matching the training environment and retaining its provenance is preferable. [S16]

A held-out spatial block or separate site is more informative than randomly splitting adjacent returns, which share local geometry and acquisition conditions. Report class-specific precision and recall, confusion matrices, unclassifiable coverage and independent terrain checks. High point-count accuracy can obscure failure in a small

but important class. A model trained on one sensor, density, season and labeling convention needs transfer testing before use elsewhere.

Code anchors: `classify/features.py` and `above.py`. Tests: `test_above`, `test_desktop_above` and `test_cli_classify`. The present classifier is not a complete semantic system for wires, poles, bridges, railway assets and all other production classes.

17 Camera projection and image colorization

Image colorization needs an oriented camera and a calibration interpreted in the stored image's coordinate system. EO Direction/Up vectors determine the camera basis used by the adapter. For a ground point X and camera center C , projection first computes camera-relative coordinates. The code's photo frame has optical direction along negative Z ; forward distance is therefore the negative third camera coordinate.

$$q = R_{cam}^T(X - C) \quad fwd = -q_z \quad (39)$$

The stored-image quarter-turn rotation is applied to the camera-plane axes before evaluating lens distortion. After the appropriate axis mapping, normalized x and y divide those axes by positive forward distance. Points behind the lens are invalid. Applying distortion in one axis convention and rotating only the resulting pixels would mishandle principal-point and tangential terms.

The implemented distortion model uses normalized radius and an Agisoft-style tangential convention. P1 and P2 are not safe to exchange with another vendor's identically named coefficients without checking the formula. The code uses three radial coefficients and two tangential terms; it does not implement every term available in a complete camera calibration package. [S18]

$$r^2 = x^2 + y^2 \quad g = 1 + K_1 r^2 + K_2 r^4 + K_3 r^6 \quad (40)$$

$$x' = xg + P_1(r^2 + 2x^2) + 2P_2xy \quad (41)$$

$$y' = yg + P_2(r^2 + 2y^2) + 2P_1xy \quad (42)$$

$$col = \frac{w-1}{2} + C_x + f x' \quad row = \frac{h-1}{2} + C_y + f y' \quad (43)$$

The focal length and principal-point offsets are pixels. The sidecar reader preserves those units, rejects unreadable present values, and refuses an ambiguous list of multiple calibrations rather than selecting one silently. Missing optional coefficients can legitimately mean zero. Imagery dimensions must match the calibrated grid. All-frame checks and per-image calibration provenance prevent a near-identical-looking camera role from concealing a different calibration.

The radial model can fold outside its valid field and map distant directions back into plausible pixels.

`Camera._validity_radius` samples the radial mapping over a finite range, finds where monotonic increase fails, and rejects a calibration that folds before reaching its own frame corner. This is an implemented guard, not an analytic proof that the full radial-plus-tangential mapping is globally invertible.

Colorization considers a configured number of nearby camera-footprint centers, projected to a single median-height plane. Among eligible unoccluded views, the most image-centered view wins. Per-photo depth grids use candidate points that project into that image, including points ultimately colored by a different image. A farther point is rejected when its range exceeds the cell minimum by more than the configured tolerance; another eligible view may then color it.

$$visible_i = [range_i \leq range_{min,cell} + \tau] \quad (44)$$

The finite candidate set and coarse depth cells are approximations. An occluder excluded from a photo's candidate set cannot affect that photo's grid, and cells crossing depth edges can produce halos. RGB is bilinearly sampled from the photograph. The current 8-to-16-bit LAS convention shifts samples left eight bits, giving 255 times 256 for

the maximum, rather than multiplying by 257. Color quality does not independently prove geometric agreement with stereo imagery.

Code anchors: `imagery/camera.py`, `colorize.py` and `job.py`; `formats/eo.py`. Tests: `test_colorize`, `test_camera_consistency` and `test_calibration_selection`. There is no implemented aerial-triangulation solver or dense multi-view reconstruction in this workflow.

18 AT referenced fitting and the SH 151 boundary

`at_fit.py` contains an experimental family of smooth Z-only corrections against reference elevations. For a target where the cloud is high, the correction observation should be reference minus cloud, so adding the predicted delta moves the cloud toward the reference. This sign is opposite the QA residual defined earlier. The module fits a median bias, a least-squares plane, or a smoothed thin-plate radial basis interpolant.

$$\delta_k = Z_{reference,k} - Z_{cloud,k} \quad Z' = Z + \hat{\delta}(E, N) \quad (45)$$

$$f(x) = \sum_i a_i \varphi(\|x - x_i\|) + p(x) \quad \varphi(r) = r^2 \log r \quad (46)$$

For the RBF, SciPy solves a smoothing system with a polynomial component and side constraints. `pyArgus` centers XY and divides by 1000 before fitting. Consequently smoothing values have meaning in that chosen coordinate normalization; they are not generic physical smoothing distances. Predictions are clipped to the observed correction range extended to include zero, and are reduced by a selected scale factor. [S19]

The model selector requires at least twelve screened targets. It compares bias, plane and smoothing values 0.1, 1, 10, 100 and 1000, each with scale factors 0.25, 0.5, 0.75 and 1. Validation combines leave-one-out prediction and five contiguous blocks ordered along the dominant XY principal axis. It minimizes the worse of the two RMSE values and accepts only if that value is below 98 percent of the uncorrected baseline.

$$score = \max(RMSE_{LOO}, RMSE_{blocked}) \quad accepted = [score < 0.98 RMSE_{baseline}] \quad (47)$$

This is a model-selection rule, not an independent final accuracy certificate. Both validation schemes participate in choosing the model; a separate untouched checkpoint set is needed to assess the selected result without selection bias. Clipping prevents unbounded amplitudes but does not establish spatial validity outside target support. A Z-only correction cannot repair horizontal misregistration, camera orientation error, occlusion, temporal change or a wrong underlying datum.

The export helper evaluates a bounded correction grid in chunks and writes a new LAS/LAZ file. It refuses an existing destination and waveform point formats, checks source metadata, preserves the intended point schema, then rereads all written point records and compares hashes against expected records. This gives unusually strong record-level export verification for that path; it should not be generalized to every writer in the project.

SH 151 remains a separate fixed-wing sensor investigation. The presence of experimental scripts, models, reports or generated surfaces is not evidence that an accepted photogrammetric surface or a successful stereo-constrained correction exists. Without independent stereo observations, a valid photogrammetric surface, or other trustworthy reference geometry, `pyArgus` cannot honestly certify agreement with the stereo block. A terrain surface derived from the LIDAR itself is not independent evidence.

Code anchors: `at_fit.py`; `reference/sh151_at_model.py`; `reference/sh151_at_preflight.py`; `reference/sh151_FINDINGS.md`. Tests: `test_at_fit`. The procedure is documented here because it exists, with experimental status rather than production endorsement.

19 Viewer cross sections and workflow state

The desktop uses one orthographic three-dimensional point viewer. It subtracts a local center before rotation, then maps rotated horizontal screen coordinates through a scale and pan. Centering protects numerical precision at

large map coordinates. The rendered pixel retains the nearest visible sampled point according to depth ordering; trajectories and contextual overlays have their own drawing behavior and are not a complete physical visibility model.

$$q = R_{view}(X - C) \quad u = sq_x + u_0 \quad v = -sq_y + v_0 \quad (48)$$

The initial overview samples points regularly across files; it is not the full cloud. Refine view scans full source files for the current view footprint and keeps a bounded sample. Zoom alone does not create a spatial index or load all hidden detail. A Shift-click point measurement identifies a displayed return, not a fitted ground elevation. View presets change the same camera; there is no second plan-only cloud viewer.

Cross-sections scan full LAS/LAZ data through a horizontal corridor. For endpoints A and B, the unit vector u follows the section and its perpendicular v points across it. Each point has station and signed cross-track distance. Inclusion is bounded by endpoint station and half the declared full width.

$$L = ||B - A|| \quad u = \frac{B-A}{L} \quad v = (-u_y, u_x) \quad (49)$$

$$station = (P - A)^T u \quad across = (P - A)^T v \quad (50)$$

$$keep = [0 \leq station \leq L] \wedge [|across| \leq width/2] \quad (51)$$

The extractor reports full matched counts while retaining a random-priority sample for display and sample export. Its cap is applied per file, so total retained points can grow with file count; this is bounded per-source sampling rather than one unconditional global cap. The result preserves file IDs, point indices, classes, line IDs, source identities, CRS and section definition. Source metadata is checked before and after scanning. Vertical exaggeration affects the section display, not stored XYZ.

The unified file sidebar is the operator's project list. Add files registers trajectories immediately even if their display confirmation is cancelled. A file being visible is distinct from its time and geometry being ready for processing. The task panel declares whole-project versus selected-cloud scope. Ground classification, surfaces and several other tasks are still single-cloud operations; a project-wide label is never a substitute for implementing a batch algorithm.

The workspace tracker stores attempts, inputs, settings, outputs, dependencies, notes and review decisions. Successful processing becomes Needs review. Accepted and Skipped require explicit decisions. Input/output metadata changes, changed settings or newer upstream attempts can mark work Outdated. An interrupted Running attempt reopens as unverified. Workspace state supports navigation and traceability, not a mathematical certificate of product accuracy.

Code anchors: viewer3d.py; sections.py; review.py; review_gui.py; workspace_state.py; workspace_gui.py. Tests: test_viewer3d, test_review_workspace, test_job_status and test_unified_workspace.

20 Persistence failures and validation

Each recorded job attempt receives a unique identifier and a separate JSON record. Large source files normally use path, size and modification time as identity; smaller critical inputs such as calibration and control may additionally be hashed by the recording adapter. The record captures settings, intended output, software versions, timestamps, results and terminal status. Which operations use the common recorder must be checked in their job adapter; coverage is not universal merely because the facility exists.

JSON is written to a temporary file, flushed, synchronized and atomically replaced. That protects the record from being partially rewritten. It is not a transaction spanning the record and every output file. A failed or cancelled operation can leave output that must not be treated as accepted. Project publication uses a temporary folder followed by publication to a new target after successful work, but specialized exporters have their own failure behavior.

Condition	Implemented or required interpretation
Missing clock or incompatible CRS	Refuse analysis or request explicit settings; do not choose silently
Unmatched or ambiguous trajectory association	Inventory diagnoses it; trajectory-based QA/alignment requires resolution
Degenerate adjustment geometry	Refuse unsupported unknowns rather than report unstable parameters
Missing geoid transformation	Refuse unsupported vertical conversion
Cancelled or failed job	Record the state; inspect any partial artifacts before reuse
Successful numerical run	Mark for review and inspect exported output independently
Changed upstream input or settings	Preserve history and identify stale downstream work

The validation runner records the exact interpreter, installed packages, platform, command exits, test outcomes, source commit, dirty state and source fingerprint before and after a run. It fails on source changes, missing results, failed tests or skipped tests requiring review. This makes a validation result more informative than an unqualified statement that tests passed.

The September 16 validation folder validation-20260916T184254Z-19000 records 390 passed, zero skipped, dependency exit 0 and packaged-self-test exit 0. It captured the final changes while the checkout was still based on d4afef9 with a dirty fingerprint; those changes were then committed as 34dd561. Its source fingerprint was unchanged during validation. The executable SHA-256 is ddf07793103183f96e8b59a066e16dce370bdcf8bdbc72330cd11ff62459e96.

A tested binary hash identifies the tested artifact but does not independently prove source-to-build provenance. The runner states that limitation. A stronger release process would capture source identity, build environment and output hashes during the build and test installation on a clean machine. Code signing, distribution license selection and dependency-license review remain release decisions rather than accomplished facts.

For mathematical changes in qa, align, classify, surfaces or formats, repository instructions require the real-data acceptance battery as well as the full unit suite. That battery needs the read-only reference drive and required geoid/native resources. No new reference run was performed while writing this manual. Documentation of old results must not be mistaken for a fresh acceptance run.

21 Recorded reference evidence

The principal historical field reference is Z:/Users/BJordan/Summerville_SS/Summerville_SS.las, not an arbitrarily similarly named file. The recorded cloud contains 15,284,332 points and four strips. Control uses explicit PNEZ order. The reference folder and scripts are documented in reference/RESULTS.md. The following figures summarize recorded September 8 experiments and are not new measurements made for this document.

Check	Recorded result	What it supports
Time join	Week 2385; 100 percent within SBET interval	Reader and time arithmetic on this export
Local control	Six supported marks; median +0.146 ft; NMAD 0.148 ft	Cross-check of the local-median pipeline; seven marks skipped
Ground overlap	Pair medians 0.000, -0.005 and +0.030 ft	Already small relative bias on delivered ground
SMRF versus delivered ground	Recall 0.9992; DTM median difference +0.111 ft	Agreement with this delivered labeling and terrain surface
Synthetic strip injection	Boresight error within 6e-6 rad with crossing line; offset error within 0.0005 ft	Recovery of constructed truth for tested geometry
Injection through real geometry	Boresight difference error within 1.9e-5 rad; offsets within 0.001 ft	Integration of time, projection, geoid, attitude and adjustment on this reference

The delivered Summerville block was already well aligned. It is useful for regression and deliberate perturbation tests, but it does not prove the ability to repair a poorly calibrated flight from a different sensor. Injection experiments supply controlled truth while retaining useful geometry; they still do not reproduce every real navigation, vegetation, time synchronization or datum error.

The ground-filter comparison also recorded point precision near 0.224 against a deliberately thin delivered ground class. Surface agreement can be informative when class conventions differ, but disagreement cannot simply be dismissed as a labeling issue in every new project. Inspect the extra points, low outliers and terrain-specific errors. The recorded canopy low_cut failure is a reminder that plausible filtering can remove the returns the project actually needs.

Use these results as reproducible targets tied to a dataset and parameter set. When an intentional algorithm change alters them, update both the acceptance expectations and the narrative results together, with a reason. Do not revise an expected number solely to make a failing test green. Archive before/after maps and independent checks so a reviewer can distinguish a justified improvement from a regression.

22 Developer operating procedure

The checkout has its own .venv. Use that interpreter for application tests, and do not install dependencies into the other development checkout. requirements-validation-win-py311.txt records the current Windows/Python 3.11 dependency baseline. Version pins help reproduction but are not a hash-locked archive of wheels, native executables, PROJ grids and all environmental settings.

From the active checkout, the ordinary validation command is:

```
& ./venv/Scripts/python.exe -m devtools.validate
```

When algorithm changes require field acceptance and a packaged artifact is available, use the explicit gates:

```
& ./venv/Scripts/python.exe -m devtools.validate --reference --exe ./dist/pyArgus/pyArgus.exe
```

Build from the maintained PyInstaller specification, choosing a fresh staging destination when OneDrive locks the existing distribution. The example destination below is illustrative and should be unused for that build:

```
& ./venv/Scripts/python.exe -m PyInstaller --noconfirm packaging/pyArgus.spec --distpath dist/release-staging --workpath build
```

The packaging launcher supports `--self-test`. The normal distribution must include its runtime folder and assets. Before replacing a user-facing build, verify the new build, retain a rollback copy when appropriate, and avoid overwriting an executable currently in use. A fresh build directory is preferable to recursive deletion of a computed path without checking its resolved location.

For a new numerical feature, first add the synthetic truth test at the algorithm boundary. Add malformed-input and unit tests at the file boundary. Add an orchestration test showing that GUI and CLI use the same parameters and preserve output metadata. Add a packaged smoke check when the change affects optional imports, assets, GUI construction or distribution behavior. Use serial resource measurements on representative files for performance claims.

The code review should ask whether the reference evidence is truly independent, whether any data was silently discarded, whether the units and signs are stated, whether a result is identifiable from the geometry, and whether export quantization changes the reported improvement. Review the actual written files, not only internal result objects. Every delivered artifact should lead back to its selected inputs, parameter values, program version and acceptance decision.

23 Remaining engineering work

The next stages should strengthen complete production workflows rather than merely increase algorithm count. The matrix below separates existing foundations from future additions. These are recommended engineering directions; they are not claims that the code already provides the capability.

Area	Existing foundation	Future addition and exit evidence
Sensor portability	Explicit clocks, frame checks and attachment diagnostics	Documented adapters for additional sensors with independent surveyed validation
Large adjustment blocks	Array solver with declared limits	Spatial patch indexing and sparse or block solution with measured memory and equivalent results
Editing	3D inspection and sections	Point selection, class editing, undo and reproducible edit history tested on production cases
Terrain constraints	Linear TIN and densified soft breaklines	Constrained triangulation, holes and hydro-enforcement with topology tests

Area	Existing foundation	Future addition and exit evidence
Surface and delivery formats	ASCII grids, LAS/LAZ, DXF/GeoJSON and optional COPC	Broader raster/vector delivery, metadata validation and contract-specific export checks
Classification transfer	Explainable cell features and random forest	Spatial holdouts, new sites, more classes and documented failure distributions
Imagery integration	Calibrated projection and RGB assignment	Independent geometric image residuals or validated photogrammetric reference surfaces
Uncertainty	Residuals and condition gates	Survey-weight models, covariance or resampling uncertainty and calibration sensitivity studies
Standards	Existing 2014-style helper and documented statistics	Explicit newer-standard reporting with checkpoint rules and survey uncertainty
Release engineering	Serial validation and packaged smoke checks	Build provenance, clean-machine installation, signing and selected license policy

Each addition needs a defined acceptance dataset, parameter range, test of known truth, failure behavior and an operator workflow. A long-term TerraSolid replacement is feasible as an iterative engineering objective, but feature breadth cannot be inferred from one successful reference site. The project's strongest current principle is to make assumptions and unresolved evidence visible before a numerical result becomes a deliverable.

24 Glossary

Term	Meaning in this manual
AGL	Navigation height above the local ground estimate, in a consistent elevation frame
AT	Aerial triangulation establishing camera exterior orientations and related calibration parameters

Term	Meaning in this manual
Boresight	Relative orientation between navigation/body and sensor frames; in this solver a small shared perturbation
Checkpoint	Reference observation reserved for independent evaluation rather than used to fit the correction
Control	Reference observation allowed to constrain the estimated model
Datum gauge	Constraint removing a free common translation or other undetermined reference mode
DTM and DSM	Ground terrain grid and upper-surface grid, with estimators explicitly defined in Section 15
EO	Exterior orientation of an image, including position and camera axes
HAG	Height above a supported ground surface
IRLS	Iteratively reweighted least squares, used here for robust residual weighting
LAS analysis strip	A project-derived association of returns; not necessarily one input file
NMAD	Normalized median absolute deviation, a robust scatter estimate
Observability	Whether the available geometry can determine the requested unknowns
Orthometric height	Elevation relative to a gravity-related vertical reference, distinct from ellipsoidal height
RBF	Radial basis function interpolator, here used in an experimental Z correction field
SMRF	Simple Morphological Filter family for ground classification
TIN	Triangulated irregular network; the current breakline treatment is soft rather than constrained

25 Sources and principle attribution

External references were checked while preparing this September 2026 snapshot. Library documentation can describe versions newer than the installed environment; the repository code and recorded package versions govern the implementation. No proprietary source code was used to establish the equations below. The links support principles, specifications and library behavior, not general approval of pyArgus results.

Specifications coordinates and standards

[S01] ASPRS. LAS Specification 1.4 Revision 15, July 9, 2019. Source for LAS coordinate scales, offsets, point records and GPS-time semantics. This is the cited specification baseline, not a claim that Revision 15 remains the latest publication. https://www.asprs.org/wp-content/uploads/2021/04/LAS_latest.pdf

[S02] PDAL documentation. readers.sbet. Source for the SBET reader format context. pyArgus implements its own explicit 17-double parser and separately assumes a geographic datum in its CRS adapter. <https://pdal.io/en/stable/stages/readers.sbet.html>

[S03] Terrasolid. TerraScan trajectory binary files. Public binary layout for TSCANTRJ version 20010715. Used as a file specification, not as an adjustment algorithm. <https://terrasolid.com/guides/tscanuav/terrascan-trajectory-binary-fi.html>

[S04] NIST. US Survey Foot and revised unit definitions. Exact survey-foot and international-foot relationships and legacy-use context. <https://www.nist.gov/pml/us-surveyfoot>

[S05] NOAA National Geodetic Survey. Converting GPS Height into NAVD88 Elevation with the GEOID96 Geoid Height Model. Cited for the fundamental height relation $h=H+N$, not as a recommendation to use the older geoid model for current projects. https://www.ngs.noaa.gov/PUBS_LIB/gislis96.html

[S06] pyproj documentation. Transformer. Source for explicit axis order, transformation options and refusal of ballpark operations. <https://pyproj4.github.io/pyproj/stable/api/transformer.html>

[S08] ASPRS. Standards Committee archive, including Positional Accuracy Standards for Digital Geospatial Data, Edition 1, 2014. Historical standard associated with the current checkpoint helper. <https://old.asprs.org/committees/standards-committee>

[S09] ASPRS. Approval of Edition 2 Version 2, June 28, 2024. Establishes changes from earlier accuracy reporting, including the treatment of 95-percent confidence metrics and survey checkpoint accuracy. <https://old.asprs.org/archives/asprs-approves-edition-2-version-2-of-the-asprs-positional-accuracy-standards-for-digital-geospatial-data-2024.html>

Adjustment filtering and numerical methods

[S07] Glira, P., Pfeifer, N., Briese, C., and Ressel, C. 2015. Rigorous Strip Adjustment of Airborne Laserscanning Data Based on the ICP Algorithm. ISPRS Annals II-3/W5, 73-80. DOI 10.5194/isprsannals-II-3-W5-73-2015. Conceptual reference for overlap constraints and sensor/trajectory adjustment; pyArgus's patch model is not an implementation of the complete published method. <https://isprs-annals.copernicus.org/articles/II-3-W5/73/2015/>

[S10] Pingel, T. J., Clarke, K. C., and McBride, W. A. 2013. An improved simple morphological filter for the terrain classification of airborne LIDAR data. ISPRS Journal of Photogrammetry and Remote Sensing 77, 21-30. DOI 10.1016/j.isprsjprs.2012.12.002. Bibliographic record verified through the authors' university research record. <https://researchconnect.suny.edu/en/publications/an-improved-simple-morphological-filter-for-the-terrain-classific/>

[S11] PDAL documentation. filters.smrf. Related algorithm family and parameter terminology; pyArgus uses its own SciPy implementation. <https://pdal.io/en/stable/stages/filters.smrf.html>

[S12] Huber, P. J. 1964. Robust Estimation of a Location Parameter. *Annals of Mathematical Statistics* 35, 73-101. DOI 10.1214/aoms/1177703732. Foundational robust-estimation reference; pyArgus's IRLS implementation and settings are defined by its own solver. <https://doi.org/10.1214/aoms/1177703732>

[S13] SciPy documentation. LinearNDInterpolator. Delaunay triangulation, linear barycentric interpolation and outside-hull behavior.

<https://docs.scipy.org/doc/scipy/reference/generated/scipy.interpolate.LinearNDInterpolator.html>

[S19] SciPy documentation. RBFInterpolator. Radial basis kernels, smoothing, polynomial terms and solvability conditions, including thin-plate splines.

<https://docs.scipy.org/doc/scipy/reference/generated/scipy.interpolate.RBFInterpolator.html>

[S20] SciPy documentation. grey_opening. Grayscale erosion followed by dilation.

https://docs.scipy.org/doc/scipy/reference/generated/scipy.ndimage.grey_opening.html

Classification imagery and persistence

[S14] Breiman, L. 2001. Random Forests. *Machine Learning* 45, 5-32. DOI 10.1023/A:1010933404324.

Foundational ensemble-classification reference. Author-hosted paper: <https://stat-www.berkeley.edu/~breiman/randomforest2001.pdf>

[S15] scikit-learn documentation. RandomForestClassifier. Actual estimator behavior and parameter definitions.

<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>

[S16] scikit-learn documentation. Model persistence. Compatibility and trusted-input limitations of pickle/joblib-based storage. https://scikit-learn.org/stable/model_persistence.html

[S17] laspy documentation. Basic Manipulation. Scaled coordinates and LAS data access; pyArgus's own writer and preservation checks determine its export guarantees. <https://laspy.readthedocs.io/en/latest/basic.html>

[S18] Agisoft. Metashape Professional Edition User Manual, version 2.2, camera-model appendix. Reference for the camera distortion convention; pyArgus's sidecar fields, half-pixel center and image-axis handling are defined by its code. https://www.agisoft.com/pdf/metashape-pro_2_2_en.pdf

Internal evidence and reproducibility locations

Repository-relative evidence: CLAUDE.md; HANDOFF.md; CODEX_WORKTREE.md; docs/CAPABILITY_ACCEPTANCE.md; docs/VALIDATION_BASELINE.md; docs/JOB_RECORDS.md; docs/QA_REVIEW.md; docs/UNIFIED_WORKSPACE.md; reference/RESULTS.md; reference/run_all.py; reference/alignment_proof.py; reference/summerville_align.py; and tests/.

Final recorded validation: reference/reports/validation-20260916T184254Z-19000/validation.json, with pytest.xml, dependencies.log, pytest.log and packaged-self-test.log in the same directory. Build log: reference/reports/consolidated-ui-build.log. The report folder is local evidence and is not guaranteed to be tracked in Git.

Reference-data locations cited in historical records: Z:/Users/BJordan/Summerville_SS for the principal Summerville acceptance cloud and trajectory; Z:/Users/BJordan/SH 151 for the separate fixed-wing investigation; and Z:/Projects/UASProjects/Connector_SR17_SR30/traj for the native TRJ import sample. Their current availability was not required or verified for this documentation task.

Appendix A Implementation map

The following catalog is generated directly from Python source at the documented snapshot. Each row names a module and its top-level functions or classes with source line numbers. Private helpers are omitted from the symbol list, although they remain part of the implementation. The catalog is a navigation aid, not a stability promise for a public API. It includes adapters and desktop modules as well as numerical methods so a maintainer can trace the complete workflow.

Core modules

Module	Top level implementation anchors
pyargus/core/georef.py	ground_points L24
pyargus/core/gridding.py	grid_edges L12; cell_indices L21; min_grid L32; max_grid L41; mean_grid L50; inpaint_nearest L62; coverage_mask L84; bilinear_sample L99
pyargus/core/rotation.py	matrix L19; matrices L31; angles L52
pyargus/core/units.py	usft_to_m L14; m_to_usft L18; ift_to_m L22; m_to_ift L26

Formats modules

Module	Top level implementation anchors
pyargus/formats/breaklines.py	read_breaklines L6
pyargus/formats/control.py	read_control_csv L17; read_control_csvs L49
pyargus/formats/copc.py	find_pdal L50; write_copc L74
pyargus/formats/crs.py	sbt_to_map L34
pyargus/formats/dxf.py	write_contours_dxf L20
pyargus/formats/eo.py	read_eo_csv L36; rotations_from_direction_up L93; camera_tag L135
pyargus/formats/geojson.py	write_contours_geojson L14; read_breaklines_geojson L31
pyargus/formats/las.py	cloud_info L97; iter_points L128; read_points L145; stream_update L186; copc_query L275; split_by_strip L333

Module	Top level implementation anchors
pyargus/formats/sbet.py	read_sbet L30; week_alignment L46; interpolate L75
pyargus/formats/trajectory.py	is_trj L6; read_times L11; validate_trj_time L20; match_times L24; load_alignment L35
pyargus/formats/trj.py	Trajectory L20; read_trj L44

Qa modules

Module	Top level implementation anchors
pyargus/qa/checkpoints.py	VerticalAccuracy L20; vertical_residuals L28; LocalComparison L48; local_median_residuals L56; robust_summary L84; asprs_vertical L94
pyargus/qa/control_by_strip.py	gather_near_marks L26; Cell L98; ControlByStrip L108; decompose L163
pyargus/qa/density.py	density_grid L11; density_grid_streamed L25; scan_extent L74
pyargus/qa/overlap.py	StripDz L19; strip_dz L41
pyargus/qa/raster.py	encode_png L33; write_png L45; write_world_file L51; grid_to_image L65; diverging_rgba L70; sequential_rgba L88
pyargus/qa/report.py	generate L28

Align modules

Module	Top level implementation anchors
pyargus/align/attach.py	AttachResult L38; bundles_from_cloud L49; apply_corrections L151
pyargus/align/bundles.py	StripBundle L20; corrected_xyz L68
pyargus/align/drift.py	DriftModel L24; nodes_for L43; bracket L58

Module	Top level implementation anchors
pyargus/align/patches.py	Correspondences L30; correspondences L47; ControlObservations L143; control_observations L159
pyargus/align/solve.py	AlignmentResult L90; solve_alignment L170

Classify modules

Module	Top level implementation anchors
pyargus/classify/above.py	AboveModel L36; train L45; predict L78; save L82; load L95; classify_above L128
pyargus/classify/features.py	eigen_shape L27; point_features L79
pyargus/classify/ground.py	GroundResult L44; smrf L76; GroundSurface L120; ground_surface L137; classify_against L170; confusion L185
pyargus/classify/job.py	candidates L44; labels L58; project_extent L83; box_reader L95; classify_ground_whole L152; classify_ground_tiled L219
pyargus/classify/tiles.py	resolve_plan L74; required_halo L100; global_edges L116; Tile L131; plan_tiles L139; plan_summary L173; outside L198; window_points L213; tiled_surface L243

Surfaces modules

Module	Top level implementation anchors
pyargus/surfaces/contours.py	ContourLine L21; contour_levels L28; contour_grid L137; smooth_chaikin L161
pyargus/surfaces/dtm.py	dtm_grid L16; dsm_grid L31; write_esri_ascii L46
pyargus/surfaces/tin.py	Tin L20; densify_polyline L52; build_tin L75

Imagery modules

Module	Top level implementation anchors
pyargus/imagery/camera.py	Camera L56; read_cal L179; find_cal L234
pyargus/imagery/colorize.py	find_images L88; load_rgb L123; check_image_sizes L129; bilinear L148; Plan L159; footprint_centers L230; image_batches L244; depth_grid L263; plan_colorization L277; apply_plan L412; prepare L475; colorize L527
pyargus/imagery/job.py	resolve_cameras L25; check_datum L98; colorize_cloud L132

Desktop modules

Module	Top level implementation anchors
pyargus/analysis_records.py	plain L11; defaults L26; analysis_job L32; finish L57; alignment_result L62; project_settings L77
pyargus/at_fit.py	predict L6; choose_model L23; correction_grid L46; evaluate_grid L58; export_cloud L64
pyargus/cli.py	build_parser L662; main L924
pyargus/gui.py	apply_theme L53; apply_branding L86; path_row L116; preview_image L143; preview_view L154; preview_photo L180; find_pylynceus L194; pylynceus_command L213; StageRunner L228; cancelled_before L324; QaStage L374; ClassifyStage L448; AboveStage L502; DtmStage L614; ContourStage L674; AlignStage L761; ColorizeStage L894; Application L991; main L1319
pyargus/job_manifest.py	identity L14; utc_now L25; JobRecord L29
pyargus/large_qa.py	percentile L20; Store L29; render_tiles L98; write_report L124; qa L216
pyargus/project.py	TrajectoryInput L19; Project L27; Cancelled L71; Track L151; LoadedProject L216; load L226; qa L501; align L512

Module	Top level implementation anchors
pyargus/project_cli.py	register L6
pyargus/project_gui.py	ProjectWindow L13; open_project L327
pyargus/review.py	load_review L7; review_rows L29; cloud_layers L87
pyargus/review_gui.py	Plot L9; ReviewWorkspace L90; open_review L383; ViewerPlan L390
pyargus/sections.py	section_coordinates L9; Section L25; extract_section L41; export_section L106
pyargus/stage_preview.py	class_colors L19; classification L39; contours L63; publish L78
pyargus/viewer3d.py	sample_clouds L10; sample_viewport L50; project_points L79; colors L88; Viewer L102; open_viewer L387
pyargus/workspace_assets.py	surface_points L5
pyargus/workspace_gui.py	Workspace L17
pyargus/workspace_state.py	stamp L16; signature L19; Tracker L28

Appendix B Test map

The test modules below are discovered from the same checkout. A named test is evidence of an implemented check, not proof that every possible behavior in the corresponding feature is covered. Use the recorded JUnit result to establish which tests ran. Domain-level grouping is inferred from filenames and is provided only to aid navigation.

Test file	Collected function definitions in source
test_above.py	10 definitions; parametrization may produce more cases
test_align.py	10 definitions; parametrization may produce more cases
test_analysis_records.py	9 definitions; parametrization may produce more cases
test_at_fit.py	3 definitions; parametrization may produce more cases
test_attach.py	8 definitions; parametrization may produce more cases
test_calibration_selection.py	4 definitions; parametrization may produce more cases
test_camera_consistency.py	5 definitions; parametrization may produce more cases
test_checkpoints.py	7 definitions; parametrization may produce more cases
test_cli_align.py	4 definitions; parametrization may produce more cases
test_cli_classify.py	5 definitions; parametrization may produce more cases
test_cli_colorize.py	4 definitions; parametrization may produce more cases
test_colorize.py	23 definitions; parametrization may produce more cases
test_contour_formats.py	5 definitions; parametrization may produce more cases

Test file	Collected function definitions in source
test_contours.py	9 definitions; parametrization may produce more cases
test_control.py	5 definitions; parametrization may produce more cases
test_control_by_strip.py	9 definitions; parametrization may produce more cases
test_copc.py	9 definitions; parametrization may produce more cases
test_crs.py	6 definitions; parametrization may produce more cases
test_density.py	4 definitions; parametrization may produce more cases
test_desktop_above.py	2 definitions; parametrization may produce more cases
test_dialog_types.py	2 definitions; parametrization may produce more cases
test_drift.py	10 definitions; parametrization may produce more cases
test_dtm.py	6 definitions; parametrization may produce more cases
test_georef.py	5 definitions; parametrization may produce more cases
test_gridding.py	4 definitions; parametrization may produce more cases
test_ground.py	5 definitions; parametrization may produce more cases
test_gui.py	23 definitions; parametrization may produce more cases
test_job_records.py	4 definitions; parametrization may produce more cases

Test file	Collected function definitions in source
test_job_status.py	2 definitions; parametrization may produce more cases
test_large_qa.py	8 definitions; parametrization may produce more cases
test_las.py	3 definitions; parametrization may produce more cases
test_overlap.py	4 definitions; parametrization may produce more cases
test_packaging_gate.py	1 definitions; parametrization may produce more cases
test_preview_controls.py	2 definitions; parametrization may produce more cases
test_project.py	19 definitions; parametrization may produce more cases
test_raster.py	6 definitions; parametrization may produce more cases
test_report.py	3 definitions; parametrization may produce more cases
test_review_workspace.py	9 definitions; parametrization may produce more cases
test_rotation.py	6 definitions; parametrization may produce more cases
test_sbet.py	9 definitions; parametrization may produce more cases
test_streaming.py	19 definitions; parametrization may produce more cases
test_tiled_ground.py	18 definitions; parametrization may produce more cases
test_tin.py	5 definitions; parametrization may produce more cases

Test file	Collected function definitions in source
test_trj.py	7 definitions; parametrization may produce more cases
test_unified_workspace.py	14 definitions; parametrization may produce more cases
test_units.py	4 definitions; parametrization may produce more cases
test_usability.py	4 definitions; parametrization may produce more cases
test_validation_manifest.py	3 definitions; parametrization may produce more cases
test_viewer3d.py	4 definitions; parametrization may produce more cases